Data Discovery in Data Lakes: Operations, Indexes, Systems

Ziawasch Abedjan
BIFOLD & TU Berlin
Berlin, Germany
abedjan@tu-berlin.de

Mahdi Esmailoghli
HU Berlin
Berlin, Germany
mahdi.esmailoghli@hu-berlin.de

Sainyam Galhotra
Cornell University
Ithaca, USA
sg@cs.cornell.edu

ABSTRACT

Data discovery has gained significant traction in the database community resulting in various discovery operations, index schemes, and discovery systems. This tutorial explores the architecture and components of data discovery systems, focusing on indexing structures and scalable algorithms for typical operations, such as join and union discovery. While giving insights into individual algorithms, we point out open challenges for holistic systems, data discovery evaluation, and discovery in federated setups.

PVLDB Reference Format:
Ziawasch Abedjan, Mahdi Esmailoghli, and Sainyam Galhotra. Data Discovery in Data Lakes: Operations, Indexes, Systems. PVLDB, 18(12): 5455 - 5459, 2025.
doi:10.14778/3750601.3750694

1 INTRODUCTION

Data lakes, whether intentionally created or emerging as a byproduct of organizational processes or open data initiatives, share a common challenge: they are often underspecified and underdocumented [1, 13]. Unlike curated data warehouses, where the schema and purpose are clearly defined, data lakes typically lack predefined objectives. Despite this, there is significant interest in leveraging these repositories for augmenting in-use datasets or pure exploratory purposes. To support such applications, data discovery as a process becomes essential, enabling users to interact with the data lake not by relying on detailed schema knowledge but by using search and matching techniques. By searching with user-provided keywords or table artifacts, users aim to uncover datasets that are relevant to their use cases.

Data discovery is currently an active field of research, as building scalable solutions to effectively capture user intent, such as augmenting a given dataset fragment, presents various challenges. A key issue is the need to index thousands to millions of tables for fast value look-ups and efficient alignment to evaluate joinability [9, 35] and unionability [19] with the user (query) table. After identifying relevant tables, assessing the usefulness of the retrieved results remains a complex problem. Often, users rely on proxy metrics, such as the downstream machine learning performance of the augmented dataset, to assess its effectiveness [7, 8, 22]. Finally, in distributed data lakes, additional constraints, such as privacy regulations and pricing structures, further complicate the process. In such cases, the usefulness and relevance of datasets must often be assessed based on available metadata and summaries, rather than direct access to the full datasets.

This tutorial is based on our previous study of data discovery use cases and our experience in building systems and algorithms that address some of its emergent challenges. Our goal is to give a structured overview of data discovery software components and to dive into technical aspects that differentiate individual prototypes and approaches. A meta-goal is to bridge the language and perspective differences on what is counted as data discovery and how discovery systems in database research can be compared. Figure 1 depicts a conceptual architecture of data discovery systems that we explore in this tutorial. The user formulates a discovery task that typically aims at enhancing an existing table or table fragment through additional features, rows, or individual values. Such tasks are then typically carried our on top of common operations, such as keyword search, join discovery, union discovery, and others that are built on top of dedicated indexes. In our tutorial, we cover the depicted architecture as follows. We first survey and discuss common data discovery use cases from industry and academia. We then present a classification of data discovery systems with layers that distinguish the locality, languages, and index structures of such systems. Locality captures the accessibility of repositories on monoliths and in distributed settings. Discovery languages can differ depending on how many different discovery tasks a system covers. Accordingly, we then dive deep into concrete index structures and how they serve individual discovery operations and languages.

Previous Tutorials: There have been similar tutorials in the past that relate to ours. In particular, the tutorial by Paton and Wu at EDBT 2024 [21] also covers various aspects of dataset search and navigation. In contrast to their tutorial, we have a strong focus on algorithmic details on how index structures and operations have evolved and can be categorized with a projection on how these approaches relate, can extend to distributed scenarios, and can be combined inside holistic systems. There has been a tutorial from the information retrieval perspective on web table extraction [32]. Our tutorial covers more recent advances and discusses the topic from the database management perspective. The tutorial on data exploration [15] is related but does not focus on data lakes.

2 TUTORIAL OUTLINE

In this tutorial, we first discuss the types of data lakes that set the motivation for building data discovery systems. We further categorize existing systems depending on their scope with regard to data locality, i.e., centralized and distributed repositories. We then detail common operations and fundamental indexing schemes in state-of-the-art that support such operations. Finally, we will discuss methods for evaluating and benchmarking data discovery systems and outline open problems and potential future research.

This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit https://creativecommons.org/licenses/by-nc-nd/4.0/ to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.
Proceedings of the VLDB Endowment, Vol. 18, No. 12 ISSN 2150-8097.
doi:10.14778/3750601.3750694

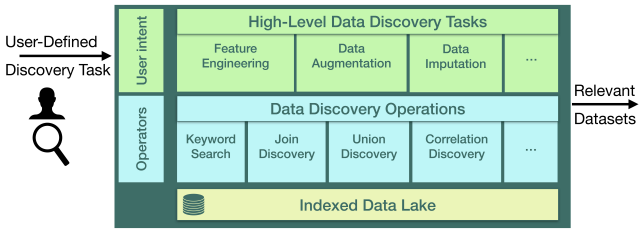


Figure 1: Vanilla data discovery system architecture

2.1 Data Lakes

With the awareness of how valuable data can be even without being currently under active usage, organizations started to accumulate datasets and make them accessible across silos. This is in contrast to the typical data warehousing approach, where handcrafted extract-transform-load pipelines are defined and documented to integrated datasets into a purposeful schema. Data lakes in contrast follow the integrate at application time paradigm. Often times the datasets not only are heterogeneous on schema level but can be subject to heterogenous data models or reside in heterogeneous storage systems. For the sake of scoping, we will focus on the algorithms and systems that have been designed for data lakes with tabular datasets.

2.2 Overview of Data discovery settings

Data discovery is a fundamental capability for organizations that manage large and often distributed collections of datasets. The need for robust data discovery systems has become even more pressing with the widespread adoption of machine learning and AI techniques, which depend critically on access to high-quality, relevant data. Numerous studies have explored different approaches to data discovery, with the optimal solution often varying based on the specific use case. This section provides an overview of the key architectural and interaction paradigms for data discovery.

Centralized vs. Federated Settings. Traditional data discovery systems typically assume *centralized access* to data, where all datasets are consolidated into a single repository, allowing the discovery engine to index and query them directly. However, this assumption is often impractical. In many cases, data is often dispersed across multiple repositories due to concerns related to data sovereignty, privacy regulations, and the high cost of data transfer.

To address these challenges, modern data ecosystems are shifting toward *federated* or *decentralized* discovery architectures. In such settings, data providers retain control over their datasets and share only metadata (or a summary of the dataset) with the discovery system. This model is common in commercial data marketplaces (e.g., Datarade, Dawex) and federated infrastructure initiatives (e.g., Gaia-X, Agora), where discovery must be performed over incomplete information, and access to data is negotiated only after identifying relevant resources.

Data Discovery Tasks and Use Cases. Users express their information needs in various ways, depending on the context and their familiarity with the data. Common interaction modalities include:

- **Keyword-based Discovery:** Users input textual queries along with optional filters. While straightforward and user-friendly, this method is often inadequate when users have nuanced requirements related to data semantics, coverage, or statistical characteristics.
- **Query-by-Example:** Users provide a small sample dataset and search for similar datasets based on structure or content. This approach is helpful when users struggle to verbalize their data needs but can illustrate them via examples.
- **Task-based Discovery:** Users specify a target task, such as training a classifier or evaluating a model, and seek datasets that are suitable for that purpose. This mode is especially relevant for machine learning workflows, where factors like class balance and feature alignment significantly impact performance.

In many real-world scenarios, users may only have a vague or evolving notion of what constitutes a “useful" dataset. Their needs may not be easily captured by keywords, examples, or tasks. To address this, recent work has focused on designing more expressive query languages that allow users to specify fine-grained predicates—such as schema constraints, statistical distributions, or semantic criteria—to better guide the discovery process.

2.3 Discovery Operations

Our tutorial will strongly focus on two major classes of operations that are common in data discovery research: Joins and Unions. note that each of the two can inhibit additional properties, such as fuzziness or conditional utility functions for the correspondingly joinable and unionable tables, as it is relevant for example for task-based discovery tasks. In our tutorial, we will define both operations and discuss the variations of each.

A join discovery operation aims at discovery tables that are joinable with a given user table. In the literature, variations, such as precalculated PK-FK relationships, single column joinability and multi-column joinability, as well as fuzzy joins have been considered. In our tutorial, we will cover the necessary algorithmic steps for each of the aforementioned variations, as well as their use cases and disadvantages. Furthermore, we will dive deep into the class of conditional join discovery approaches, where not only the joinability but also the utility of the obtained tables with regard to downstream tasks have been considered.

A union discovery operation aims at vertically expanding a table with more rows from tables inside a lake. Within this line of work, we will distinguish approaches that check for perfect unionability as in consistent with UNION in SQL as well as approaches that find partially unionable tables.

2.4 Indexing Data Lakes

An efficient index structure is necessary to support any of the aforementioned discovery operators. Index structures have long served as the foundation of traditional information systems, enabling efficient search and retrieval. In the era of large-scale data lakes and advanced data discovery techniques, these structures have evolved to meet two primary objectives: *scalability* and *customizability*. The large size and heterogeneity of data lakes require indexes that can efficiently scale to large volumes of data while supporting various

Table 1: Mapping between indexes and data discovery studies.

Index		Papers
Value (Cell)	Value → Location	[2, 4, 24, 29–31, 35]
	Location → Value	[2, 30, 35]
	Value → Class (Domain)	[24]
	Value → Embedding	[5]
Column	LSH Variations	[3, 5, 12, 14, 19, 28, 36]
	Column → Metadata	[2, 14, 24, 30, 35]
	Column Names → tables	[31]
	Column → Domain	[34]
	Column → Rank	[22, 23]
	Column → Embedding	[5, 11, 27]
Row	Row → Table	[2, 31]
Table	Table → Embedding	[31]
	Table → Metadata	[2]
Graph	V: Columns, E: Similarities	[4, 12, 16–18, 20, 26, 31, 33, 34]

custom search requirements. For instance, data discovery with the goal of benefiting a downstream machine learning (ML) task requires a task-specific similarity measure to find the most relevant data. In this example, the relevancy of a column is measured by how significantly the column correlates with a target column to be predicted in the ML task. These task-specific requirements demand customized index structures, leading to hybrid and more complex indexes.

Index structures for data discovery in data lakes can be categorized based on two key dimensions: query interface, determining the primary way users interact with the index, and search approximation, defining whether the index allows exact or approximate discovery.

Table 1 provides an overview of the state-of-the-art index structures, grouped into six categories based on their query interface: Value-based indexes, column-based indexes, row-based indexes, table-based indexes, and graph-based indexes.

Each of these index types serves a distinct purpose. Value-, column-, row-, and table-based indexes enable querying data lakes at different levels of granularity, from individual cell values to entire tables. Graph-based indexes facilitate exploratory search by capturing relationships between datasets, allowing users to navigate table connections.

Index structures can also be categorized based on whether they provide exact or approximate, i.e., fuzzy discovery. Exact indexes guarantee exact retrieval of the most relevant tables to the task at hand but may be computationally expensive. Examples of exact indexes are traditional inverted indexes. Approaches such as Josie [35] and MATE [9] benefit from exact indexes to find equi-joinable tables to a given key column. On the other hand, approximate indexes prioritize efficiency by sacrificing accuracy.

Hash- and sketch-based techniques have been widely adopted to improve scalability. For example, LSH-based indexes [6, 14, 36] are used for efficiently finding tables based on Jaccard, containment, and cosine similarities. However, significant research efforts focus on improving their accuracy and scalability, leading to variations such as LSH Ensemble [36], which reduces bias towards smaller tables, or Lazo [14], which uses OOPH sketches [25] to reduce the number of hashes required before constructing the LSH index. Sketch-based indexing techniques, such as Quadrant Count Ratio (QCR) [23], have also been employed to find correlating features to a target column.

High-dimensional vector indexes, such as HNSW, Inverted File Index (IVF), and Product Quantization (PQ), have recently been used to enable approximate data discovery based on semantic similarity between tables [11].

2.5 Evaluating Discovery Systems

One of the major challenges in data discovery research is the appropriate evaluation of proposed methods. Similar to other research problems in data integration, there is always a trade-off between resource consumption and quality of potential techniques. While measuring resource consumption and runtime is straightforward, it is often hard to set the proper criteria for the expected quality of data discovery results. In this tutorial, we want to delve into these challenges and discuss two different approaches that are currently carried out by researchers: 1) Close-world: A ground truth is created, often by fragmenting larger tables and trying to resynthesize the original tables. With this approach, metrics such as precision and recall can be calculated. 2) Open-world: Data discovery aims at finding serendipitous datasets that are useful for a downstream use case. Both approaches can bring insights into the capabilities of different techniques. In our tutorial, we want to highlight that the implication of each is different, which in turn might favor a certain class of techniques.

2.6 Future Directions

As our tutorial will show, research on data discovery has made significant progress in the past decade. Nevertheless, there are aspects that are still underexplored and impede the usage of data discovery solutions in practice. Among those, we will discuss the challenges in building holistic systems [10] that serve a variety of discovery tasks, data discovery in federated setups and under constraints as well as the expansion of existing techniques for additional modalities beyond relational tables.

3 TUTORIAL ORGANIZATION

We plan to carry-out the tutorial as an interactive lecture with live demonstration of discovery operations and systems. As several of the systems have been developed and reproduced at our labs, we can provide practical insights into several of the presented techniques. Overall, the structure as outlined in the previous section can be covered within 3 hours. We can trim the tutorial for a shorter session if needed, by focusing on fewer discovery operations.

Target audience. The topic of data discovery has a broad appeal within the data management community. We believe that anyone who is interested in data discovery, security, and privacy of data would be interested in this tutorial. More broadly, the tutorial is aimed at encouraging the DB community to work on novel problems in data discovery and make it more practical. All materials used will be released publicly, and the attendees will be given a hands-on experience to test the presented techniques.

Prerequisites. The background expected is that of an introductory course on data management. The tutorial has been carefully structured to accommodate both attendees unfamiliar with the topic and experienced participants by providing the required background knowledge and shared terminology.

GitTables: A Large-Scale Corpus of Relational Tables

MADELON HULSEBOS*, University of Amsterdam, Netherlands
ÇAĞATAY DEMIRALP, Sigma Computing, United States
PAUL GROTH, University of Amsterdam, Netherlands

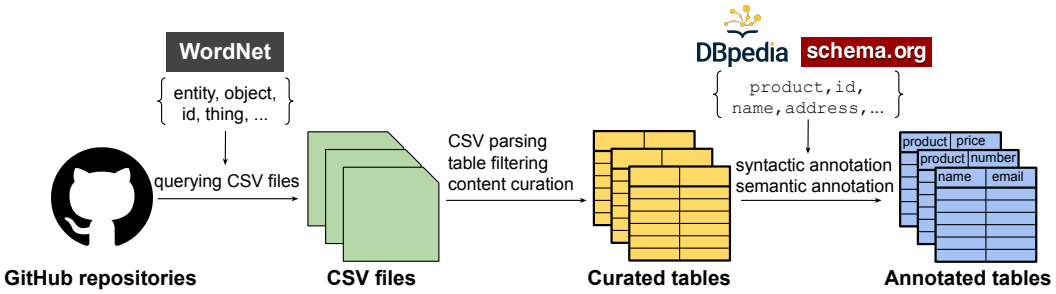


Fig. 1. The pipeline for creating GitTables consists of 1) querying CSV files from GitHub repositories based on WordNet, 2) parsing CSV files and curating tables, and 3) annotating tables with column semantics from DBpedia and Schema.org.

The success of deep learning has sparked interest in improving relational table tasks, like data preparation and search, with table representation models trained on large table corpora. Existing table corpora primarily contain tables extracted from HTML pages, limiting the capability to represent offline database tables. To train and evaluate high-capacity models for applications beyond the Web, we need resources with tables that resemble relational database tables. Here we introduce GitTables, a corpus of 1M relational tables extracted from GitHub. Our continuing curation aims at growing the corpus to at least 10M tables. Analyses of GitTables show that its structure, content, and topical coverage differ significantly from existing table corpora. We annotate table columns in GitTables with semantic types, hierarchical relations and descriptions from Schema.org and DBpedia. The evaluation of our annotation pipeline on the T2Dv2 benchmark illustrates that our approach provides results on par with human annotations. We present three applications of GitTables, demonstrating its value for learned semantic type detection models, schema completion methods, and benchmarks for table-to-KG matching, data search, and preparation. We make the corpus and code available at https://gittables.github.io.

CCS Concepts: • **Information systems** → *Relational database model*; *Information integration*; • **Computing methodologies** → *Information extraction*; *Neural networks*.

Additional Key Words and Phrases: relational tables, data management, deep learning, datasets

ACM Reference Format:
Madelon Hulsebos, Çağatay Demiralp, and Paul Groth. 2023. GitTables: A Large-Scale Corpus of Relational Tables. *Proc. ACM Manag. Data* 1, 1, Article 30 (May 2023), 17 pages. https://doi.org/10.1145/3588710

*Corresponding author (m.hulsebos@uva.nl)

Authors' addresses: Madelon Hulsebos, University of Amsterdam, Amsterdam, 1012 WX, Netherlands, m.hulsebos@uva.nl; Çağatay Demiralp, Sigma Computing, San Francisco, CA 94105, United States, cagatay@sigmacomputing.com; Paul Groth, University of Amsterdam, Amsterdam, 1012 WX, Netherlands, p.t.groth@uva.nl.

This work is licensed under a Creative Commons Attribution International 4.0 License.

© 2023 Copyright held by the owner/author(s).
2836-6573/2023/5-ART30
https://doi.org/10.1145/3588710

1 INTRODUCTION

Deep learning (DL) models, in the past decade, have dramatically improved many long-standing computer vision and natural language processing tasks [25]. The practical success of DL has also spurred interest in its applications to tasks in other domains, including data management, ranging from data cleaning to annotation [45]. To train DL models for relational data, earlier work primarily relied on corpora consisting of tables scraped from HTML pages [10] such as WDC WebTables [46], the largest table corpus to date. These models have played an important role in facilitating data-driven data management research, with Web applications in particular [7, 49].

However, tables extracted from HTML pages on the Web (Web tables) provide a skewed representation of tables in the wild especially those residing in (enterprise) databases [24, 27]. The common attribute "ID", for example, does not appear among the twenty most frequent column names of the WDC WebTables corpus, and the dimensions of Web tables are significantly smaller than typical database tables [46]. It is therefore unsurprising that models based on Web tables have limited applications beyond the Web [7]. To broaden the impact of data-driven data management research, we need new table collections complementing existing corpora with tables resembling typical database tables.

To address this demand, we introduce GitTables: a corpus with 1M relational tables extracted from CSV files in GitHub repositories. We will keep the corpus growing to have at least 10M tables to facilitate the extension of deep transfer learning to the relational domain, analogous to how large corpora of natural language have stimulated pretrained language models like BERT [16] and GPT [6]. We expect GitTables to stimulate similar progress in data management tasks like data search and preparation.

Besides having representative tables, many data management applications benefit from understanding the semantics of table columns. An intelligent data exploration system, for example, could recommend a map chart to visualize two data columns representing countries and sales revenues. Upon encountering missing values, the system could also automatically fill the gaps using a knowledge base. The system could, for instance, impute "France" in an empty cell in a column with country values, if the neighboring cell in a column with capital cities is "Paris".

To facilitate such functionalities, we enhance table columns with semantic types from DBpedia and Schema.org (e.g. `address`) using a syntactic as well as semantic annotation method (an example in Figure 2). Each semantic type is also associated with the expected atomic data type (e.g. `string` and `number`), a description, and hierarchical type relations. These semantically rich annotations together with DL present a unique opportunity to learn table semantics as demonstrated with TURL [15] and TaBERT [48].

Our analysis of GitTables confirms the different nature of the tables: the tables have significantly larger dimensions (rows and columns). A machine learning model for detecting data shift between GitTables and Web tables accurately distinguishes table columns from each source corpus. This reflects the structurally different content of these corpora. The semantic type distribution also deviates significantly from the semantic distribution of Web tables, illustrating its distinctive topical coverage. We make GitTables available through https://gittables.github.io. We contribute:

(1) GitTables, a new large-scale corpus of 1M tables. To the best of our knowledge, GitTables is the first large-scale relational table corpus with topical coverage and content structurally different than tables extracted from HTML pages.
(2) A scalable column-annotation method using distant-supervision. We annotate the columns in GitTables with semantics consisting of semantic types, atomic data types, hierarchical relations, and descriptions, making GitTables the largest annotated table corpus to date.

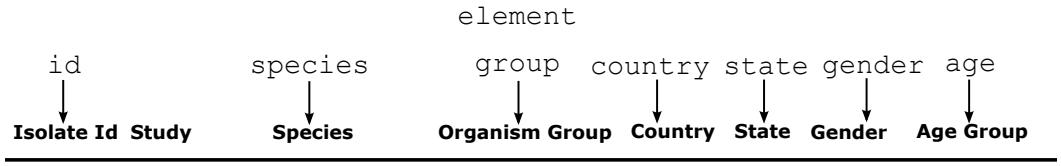


Fig. 2. Snippet of an annotated table in GitTables. Annotations are provided with confidence scores. For example, the confidence of the type `species` for the column "Species" is 1, while `element group` for "Organism Group" is 0.7. These scores enable filtering based on the intended use case.

Table 1. Existing large-scale relational table corpora in comparison with GitTables which has considerably higher dimensions, resembling actual tables in databases. We report statistics related to relational tables, leaving out, for example, entity tables.

Name	Table source	# tables	Avg # rows	Avg # cols
WDC WebTables [26]	HTML pages	90M	11	4
Dresden Web Table Corpus [17]	HTML pages	59M	17	6
WikiTables [3]	Wikipedia tables	2M	15	6
Open Data Portal Watch [29]	CSVs from Open Data portals	107K	365	14
VizNet [20]	WebTables, Plotly, i.a.	31M	17	3
GitTables	**CSVs from GitHub**	**1M**	**142**	**12**

(3) Three applications demonstrating the value of GitTables: 1) we train a semantic type detection model and obtain high prediction accuracy, 2) we use GitTables as a resource for schema completion methods in typical database contexts, and 3) we present a starting point for benchmark datasets for typical data management tasks.

2 RELATED WORK

Web initiatives such as Common Crawl, Wikipedia, and Open Data have been cost-effective resources for curating unstructured and structured data at scale [3, 31, 32]. Below we discuss large-scale table corpora sourced from these initiatives and review prior work annotating column semantics of tables sampled from these corpora.

2.1 Large-scale table corpora

WDC WebTables and Dresden Web Table Corpus. These Web table corpora [17, 26] extract tables from HTML pages in the Common Crawl corpus as inspired by [8]. They provide an abundance of relational tables ranging from 59M to 90M and have been instrumental in advancing applications like table augmentation and integration [7, 49]. However, Web tables generalize poorly due to their small dimensions and different content [24].

WikiTables. To provide high-quality tables with semantics that are easier to detect than those of arbitrary Web tables, WikiTables extracts approximately 2M tables from Wikipedia [3]. This corpus is primarily suitable for tasks such as question answering that rely on the quality of the table contents. Unsurprisingly, the tables in WikiTables are just as small as those in WDC WebTables.



LakeBench: A Benchmark for Discovering Joinable and Unionable Tables in Data Lakes

Yuhao Deng
dyh18@bit.edu.cn
Beijing Institute of Technology

Chengliang Chai
ccl@bit.edu.cn
Beijing Institute of Technology

Lei Cao
lcao@csail.mit.edu
University of Arizona/MIT

Qin Yuan
yuanq1020@gmail.com
Beijing Institute of Technology

Siyuan Chen
siyuanchen@bit.edu.cn
Beijing Institute of Technology

Yanrui Yu
yanruiyu@bit.edu.cn
Beijing Institute of Technology

Zhaoze Sun
sunzhaoze@bit.edu.cn
Beijing Institute of Technology

Junyi Wang
wangjunyi@bit.edu.cn
Beijing Institute of Technology

Jiajun Li
lijiajun@bit.edu.cn
Beijing Institute of Technology

Ziqi Cao
caoziqi@bit.edu.cn
Beijing Institute of Technology

Kaisen Jin
jks@bit.edu.cn
Beijing Institute of Technology

Chi Zhang
zhangchi315@bit.edu.cn
Beijing Institute of Technology

Yuqing Jiang
jiangyuqing@bit.edu.cn
Beijing Institute of Technology

Yuanfang Zhang
yfzhang@bit.edu.cn
Beijing Institute of Technology

Yuping Wang
wyp_cs@bit.edu.cn
Beijing Institute of Technology

Ye Yuan
yuan-ye@bit.edu.cn
Beijing Institute of Technology

Guoren Wang
wanggr@bit.edu.cn
Beijing Institute of Technology

Nan Tang
nantang@hkust-gz.edu.cn
HKUST (GZ)

ABSTRACT

Discovering tables from poorly maintained data lakes is a significant challenge in data management. Two key tasks are identifying joinable and unionable tables, crucial for data integration, analysis, and machine learning. However, there's a lack of a comprehensive benchmark for evaluating existing methods. To address this, we introduce LakeBench, a large-scale table discovery benchmark. It evaluates effectiveness, efficiency, and scalability of table join & union search methods. With over 16 million real tables, LakeBench is **1,600X** larger than existing datasets and **100X** larger in storage size. It includes synthesized and real queries with ground truth, totaling more than 10 thousand queries – **10X** more than used in any existing evaluation. We spent over 7,500 human hours labeling these queries and constructing diverse query categories for thorough evaluation. Our benchmark thoroughly evaluates state-of-the-art table discovery methods, providing insights into their performance and highlighting research opportunities.

PVLDB Reference Format:
Yuhao Deng, Chengliang Chai, Lei Cao, Qin Yuan, Siyuan Chen, Yanrui Yu, Zhaoze Sun, Junyi Wang, Jiajun Li, Ziqi Cao, Kaisen Jin, Chi Zhang, Yuqing Jiang, Yuanfang Zhang, Yuping Wang, Ye Yuan, Guoren Wang, and Nan Tang. LakeBench: A Benchmark for Discovering Joinable and Unionable Tables in Data Lakes . PVLDB, 17(8): 1925-1938, 2024.
doi:10.14778/3659437.3659448
PVLDB Artifact Availability:
The source code, data, and/or other artifacts have been made available at https://github.com/RLGen/LakeBench.

This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit https://creativecommons.org/licenses/by-nc-nd/4.0/ to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.
Proceedings of the VLDB Endowment, Vol. 17, No. 8 ISSN 2150-8097.
doi:10.14778/3659437.3659448

1 INTRODUCTION

Many governments have acknowledged the importance of providing open access to their data for the public good [30]. However, these open datasets are typically published in formats like image or JSON that don't fit neatly into relational databases. To make sense of this poorly maintained data, data scientists have to go through a pipeline of data curation tasks to discover, merge, and clean data [5, 9–11, 31]. As the first step of this pipeline, data discovery that finds relevant tables is critical for data scientists to effectively and efficiently conduct their work.

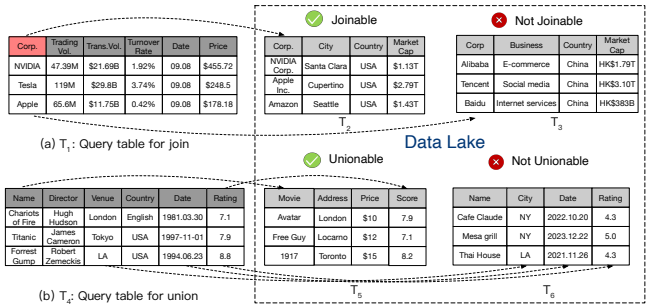


Figure 1: Table Discovery in Data Lake.

Table discovery from data lakes can be generally classified into two categories: keyword-based search [4, 16, 19, 20, 40] and table-based search [3, 13, 14, 17, 24, 32, 45, 46]. Keyword-based search over

tables often relies on the metadata of tables in the lake to identify tables or tuples relevant to the given keywords. However, in the data lake scenario, the metadata is often poorly maintained compared to the database management system (DBMS). Consequently, the incompleteness and inconsistency of metadata in the data lake become bottlenecks for keyword-based search. Thus in this work, we focus on the table-based search that can be further categorized into table join search and table union search. Recently, table-based search has attracted much attention in data management field to augment data instances [7, 27, 43] or attributes [6, 8, 26, 28, 39]. This benefits many downstream applications such as machine learning based data analysis.

Existing Works. To the best of our knowledge, no work has specifically focused on constructing a benchmark for the thorough evaluation of the existing table search methods. Although some works that target inventing new table search methods have done a great job in releasing their datasets and queries used in their evaluation, the scale of their datasets, the number of queries, and the diversity of the workload are not sufficient to evaluate the effectiveness, scalability, and efficiency of the table search methods in diverse scenarios. In particular, TUS [32] and Santos [24] evaluate their *table union search* methods with 1,000 queries on *small* data lakes which contain at most 11,090 tables – in total 11GB data, far from being sufficient to evaluate the scalability of the search algorithms and simulate the diverse real applications. Josie [45], targeting *table join search*, releases 1,000 queries to evaluate if their method is able to efficiently find the tables that have exact overlap with the query tables. In this setting, they do not have to label their queries, e.g., manually discover the tables joinable with the query tables. Therefore, these queries cannot be used to evaluate the effectiveness of other table join search methods that consider the semantic similarity of the data.

Design Goals. To fill this gap, in this work we aim to construct a comprehensive benchmark for table discovery, following the design goals below:

(1) *Scalability.* To thoroughly evaluate the efficiency and scalability of different methods, the benchmark has to involve data lakes of various sizes, especially large-scale ones. The size of the data lake is measured from two aspects: the storage size and the number of total columns. The storage size is highly related to the average table size and the number of tables. The number of total columns matters as well because almost all table discovery algorithms index and search over a large number of columns.

(2) *Variety.* The performance of table discovery largely relies on the characteristics of the datasets and queries. In particular, one algorithm might perform differently on different categories of queries. For example, given a query, a semantic-aware algorithm will show clear advantages if the table semantic matters the most in determining its unionable tables. Hence, plenty of diverse queries are critical to thorough evaluation.

(3) *Comprehensive comparison of solutions.* Understanding the pros/cons of existing table search methods and revealing research opportunities, requires a reliable implementation of all these methods, a thorough and fair experimental evaluation, and a deep analysis of the results.

Our Proposal. Guided by the above three design goals, we construct a benchmark, LakeBench– the first of its kind that supports the thorough evaluation of table union & join methods.

To achieve the first design goal, we collect over 1TB data – **100X** larger than the data lakes used in existing works, and the number of columns is up to **100 million**, on which we evaluate different table union/join search methods. Moreover, we label a sufficient number of diverse queries (more than 10 thousand) – **10X** more than any other works. To this end, we organize a team of 25 graduate students and spent more than *7,500 human hours* on labeling the queries. Because it is prohibitively expensive to manually identify tables in the data lake that are joinable/unionable to a given query, we design a candidate generation strategy to prune the tables that are most unlikely to be the ground truth.

To meet the second design goal, we construct data lakes with diverse characteristics by collecting tables from two data sources, namely OpenData [1] and WebTable [2]. WebTable has a large number of tables (more than 10^8) but each table is small. In contrast, OpenData consists of large tables (on average 1GB in size per table) but has fewer tables than WebTable. Besides, we generate both synthetic (splitting from large lake tables) and real queries (directly sampling from the data lake), which have different properties. We further categorize these queries at finer granularity w.r.t. the key factors that determine the quality of table search.

For the third design goal, we implement many key techniques such as column embeddings, approximate nearest neighbor (ANN) search index, etc., which are the building blocks of these approaches, ensuring that we are able to evaluate the scalability and accuracy of various approaches on different datasets and mostly reuse the code base. More importantly, we evaluate these approaches over the aforementioned fine-grained query categories, so as to reveal multiple insights w.r.t. the performance of different approaches over different categories. In addition, our analysis of the time and space complexity of the search algorithm connects the experiment results to theory, guiding our discussions on the results. In short, LakeBench makes it possible to thoroughly evaluate and understand existing table search methods.

Contributions. We make the following contributions:

(1) We build a comprehensive benchmark, LakeBench, for table discovery, featuring large-scale datasets and a sufficient number of diverse queries. Besides, it addresses critical limitations in existing benchmarking datasets. To be specific, *i*.) The lack of a specific dataset for joinable search with ground truth; *ii*.) The number of ground truth tables for individual queries is small, particularly notable in TUS and SANTOS, thus problematic in accurately assessing recall; *iii*.) Relying heavily on automatically generated ground truth, which may simplify the task and potentially miss ground truth because of lacking sufficient human annotation.

(2) We collect real tables (more than 1 TB in size) from multiple sources, which consist of 4 data lakes with various storage sizes and different numbers of table columns.

(3) We create 10,000+ table queries covering different characteristics and spend 7,500+ human hours on accurately labeling them.

(4) We thoroughly evaluate and analyze the state-of-the-art table join/union search approaches using these created queries on the benchmark datasets. Based on that, we reveal multiple insights and research opportunities. For example, we observe that although

the methods that fine-tune pre-trained language models in general show an advantage over other methods, their accuracy is still not very satisfactory in some cases. Therefore, designing better fine-tune strategies or fine-tuning large language models (LLM) might be a promising direction to go in the future.

2 PRELIMINARY

2.1 Problem Definition

We formally define the two table discovery tasks in a data lake.

Table Join Search. Suppose that a data lake $\mathcal{T}$ contains a large set of tables $\mathcal{T} = \{T_1, T_2, ..., T_N\}$, where each table $T_i, i \in [1, N]$ has n_i rows (tuples), m_i columns (attributes) and each cell value is denoted by c_{ij}. Given a query table T_q^J, as well as a specific column C_q^J of T_q^J, table join search is to find the target tables that can be joined with T_q^J on C_q^J, and we note the measure of the relevance score between two columns as $R(C_q^J, C_t)$, where C_t is a column of a target table $T_t \in \mathcal{T}$. The higher the score, the more likely the two columns can be joined (In this case, we can say that the two columns or two tables can be joined interchangeably), *i.e.*, $R(C_q^J, C_t)$ can also be represented as $R(C_q^J, T_t)$. Note that different methods have different criteria to compute the score, like the number of overlaps and/or semantic similarity. The formal definition is as follows.

Definition 1 (Top-k Table Join Search). Given $\mathcal{T}$, a query table T_q^J, the specific column C_q^J and a parameter k, top-k table join search aims to retrieve a subset $\mathcal{T}_q \subset \mathcal{T}$, $|\mathcal{T}_q| = k$ such that $\forall T \in \mathcal{T}_q$ and $\forall T' \in \mathcal{T} \setminus \mathcal{T}_q, R(C_q^J, T) > R(C_q^J, T')$.

Remark. Given a target table T_t, there may exist multiple columns that can be joined with T_q. In this case, we take the one with the highest score as the target column, *i.e.*, $C_t^J = \arg\max_{C \in T_t} R(C_q^J, C)$. Also, we focus on the two-way join rather than multi-way joins [15].

Table Union Search. Given a query table T_q^U, table union search aims at finding the top-k unionable tables from the data lake $\mathcal{T}$. At a high level, similar to table join search, table union search highly relies on the unionbility of the columns, *i.e.*, a pair of unionable tables should have multiple pairs of columns that can be unioned. Like table join search, column unionbility also takes the overlaps and/or semantics between columns into consideration. To be specific, given a target table T_t, we can measure the relevance of each pair of columns, *i.e.*, $R(C, C'), C \in T_q^U, C' \in T_t$, and compute a table-level relevance score as $R(T_q^U, T_t)$.

Definition 2 (Top-k Table Union Search). Given $\mathcal{T}$ and a query table T_q^U, top-k table union search aims to retrieve a subset $\mathcal{T}_q \subset \mathcal{T}$, $|\mathcal{T}_q| = k$ such that $\forall T \in \mathcal{T}_q$ and $\forall T' \in \mathcal{T} \setminus \mathcal{T}_q, R(T_q^U, T) > R(T_q^U, T')$.

More specifically, given a query table with a user-specified column, table join search finds the target tables that can be joined with the query table on the user-specified column.

Example 1 (Table join search.). *As shown in Figure 1(a), given a query table (T_1) and a specified column* `Corporation`*, Table T_2 in the data lake is joinable with T_1, i.e., the first attribute of T_2 can be joined with the specified column because i) the column names match and ii) a number of cell values have fuzzy overlaps (e.g., Apple and Apple Inc.). Joining these two tables together produces a table with richer features, which is beneficial when used to train a machine learning model to for example predict the stock* `Price` *of the companies in the table. On the other hand, although in T_3, the column name and the contents of its first attribute are semantically similar to the specified column, it is not joinable because there is no overlapping value.*

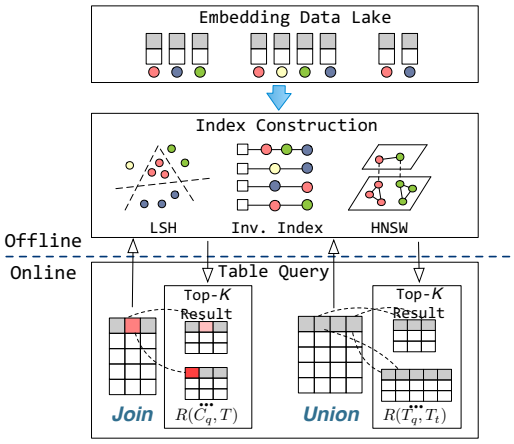


Figure 2: Table Discovery in Data Lake.

For table union search, given a query table, it aims to find target tables that are unionable with the query.

Example 2 (Table union search.). *As shown in Figure 1(b), given a query table (T_4), T_5 can be unioned with T_4 because they are all about the information of movies and two pairs of attributes are highly correlated (T_4.*`Name` *v.s. T_5.*`Movie` *and T_4.*`Rating` *v.s. T_5.*`Score`*). However, when it comes to T_6, although T_6 has three attributes (T_6.*`City`*, T_6.*`Date` *and T_6.*`Rating`*) aligned respectively with attributes (T_4.*`Venue`*, T_4.*`Date` *and T_4.*`Rating`*) of T_4, T_6 is not unionable with T_4. This is because the two tables are not in the same context: T_4 is about movies, while T_6 is about restaurants.*

Examples 1 & 2 show that effectively discovering joinable and unionable tables faces multiple common challenges, including (1) the (semantic) schema similarity of columns, (2) the overlap between columns, and (3) the contextual information across multiple columns in a table. Moreover, as a data lake is typically large-scale, designing efficient and scalable discovery algorithms is of high complexity. Although many approaches have been proposed to address the above problem from different perspectives, these approaches are yet to be thoroughly evaluated due to the lack of a comprehensive benchmark.

Remark. Note we return the top-k results rather than asking users to set a cut-off threshold w.r.t. the relevance score and returning all tables that have a relevance score higher than this threshold. This is because setting a threshold appropriate to the given query is harder than setting the k parameter. Therefore, almost all [3, 14, 17] existing works focus on retrieving the top-k results.

2.2 Table Discovery Process

Most of the table discovery methods consist of the following three modules: data lake embedding, index construction, and online table

Rottnest: Indexing Data Lakes for Search

Ziheng Wang*+ Stanford University
Sasha Krassovsky* Anthropic PBC
Conor Kennedy*, Alex Aiken Stanford University
Weston Pace LanceDB, Inc.
Rain Jiang*, Huayi Zhang, Chenyu Jiang, Wei Xu Bytedance, Inc.

***Abstract*—Data lakes have become widely popular in managing enterprise data. Their widespread integration with query engines has allowed them to displace specialized data warehouses as the single source of truth for enterprise data. While the columnar storage format and block min-max indices allow query engines to achieve competitive performance on relational data analytics queries, they are not yet suitable for other search-oriented queries like full text and vector nearest neighbor search. We present Rottnest, a general system that builds additional lightweight indices on top of data lakes. We show that our system is more cost efficient compared to un-indexed data lakes or specialized databases across several orders of magnitude of total query loads and operating time horizons.**

I. Introduction

Data lakes have become mainstream in analytical data processing. They typically consist of Parquet files on object storage such as AWS S3 organized by a table format like Delta Lake, Iceberg or Hudi [1]–[5]. The Parquet files are queried on-demand with engines such as Trino, Databricks, Snowflake, or DuckDB [6]–[10]. For OLAP SQL workloads, the performance of such query engines is often better than specialized data warehouses [11], [12]. The widespread support of Parquet among data science and machine learning tools have made data lakes ideal for the analytical "single source of truth" in modern enterprise data stacks.

However, data lakes are still unsuitable for emerging workloads that are **search-oriented**. These workloads, such as high-cardinality time series, text, and embedding analytics typically need to quickly drill down to a very small subset of the data and perform complex aggregations on this subset. In most cases, the minimal indexing available in current Parquet-based data lakes do not allow efficient evaluation of the filter conditions (e.g. UUID match, text regex, approximate nearest neighbors), making these workloads inefficient for query engines like SparkSQL or Trino [6], [13]. As a result, the lakehouse paradigm breaks for these workloads – organizations have to duplicate their data to a specialized data management system like Clickhouse, ElasticSearch or Qdrant to perform efficient search analytics, as shown in Figure 1.

Modern data lakes store data primarily as immutable Parquet files in object storage, optimized for large-scale analytics. Data lakes like Delta Lake and Apache Iceberg added transactional support to data lakes by maintaining lightweight metadata files alongside the Parquet files [4], [5], [14]. These systems achieve ACID properties efficiently through a combination of metadata logging and background compaction operations - transaction logs track file-level changes while periodic compactions merge small files to optimize read performance. Building on these principles, we explore if we could support novel search workloads through additional lightweight index files on top of transactional data lakes. We build Rottnest, a system that augments data lakes with external indices resident on object storage for workloads such as high cardinality analytics, full text search, and vector embedding search.

Although individual components of Rottnest's novel architecture such as inverted indices and LSM-style compaction are well studied, our key insight is that combining them enables efficient search over data lakes with low query cost and storage overhead. Rottnest's architecture is enabled by a novel consistent-on-demand indexing protocol that can be bolted on to any data lake and object storage backend. We also propose a novel evaluation methodology that demonstrates Rottnest's effectiveness across a wide range of operational scenarios.

Just as data lakes provide "good enough" transactional semantics for offline big data workloads without replacing OLTP databases, Rottnest provides "good enough" search capabilities for offline workloads without aiming to replace specialized search systems in latency-sensitive online serving. We believe it can be the most compelling option for most offline workloads such as historical log analytics and LLM pretraining data exploration.

In summary, we make three contributions:

1) A novel bolt-on, consistent-on-demand protocol for concurrent index maintenance. Our protocol, with formal correctness proofs, enables consistent indexing even as the underlying data lake undergoes operations like compaction and deletion. Unlike existing alternatives, it only relies on strong read-after-write consistency, making it compatible with all major cloud providers.
2) A new system architecture to support search-oriented workloads over data lakes using our lazy indexing protocol that combines in-situ querying of Parquet files directly and object-store native indices to deliver low query cost and low storage overhead.
3) A novel evaluation framework using physics-inspired *phase diagrams* to analyze total cost of ownership (TCO) across different query loads and time horizons. This methodology reveals insights about system trade-offs that would be difficult to obtain through traditional performance evaluations, and can be applied to evaluate other cloud data systems where TCO is the relevant metric.

* Equal contribution. + Currently at Anthropic PBC.

2025 IEEE 41st International Conference on Data Engineering (ICDE) | 979-8-3315-3603-9/25/$31.00 ©2025 IEEE | DOI: 10.1109/ICDE65448.2025.00139

2375-026X/25/$31.00 ©2025 IEEE
DOI 10.1109/ICDE65448.2025.00139

Authorized licensed use limited to: The University of Utah. Downloaded on January 17,2026 at 03:38:49 UTC from IEEE Xplore. Restrictions apply.

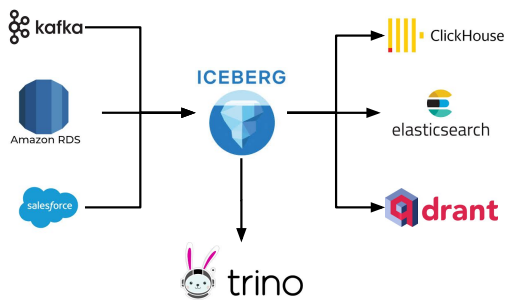


Fig. 1. Typical enterprise data stack. For workloads where directly querying the data lake with a query engine like Trino is too inefficient, a specialized system like Clickhouse, ElasticSearch or Qdrant is used.

II. Motivations

In this section, we describe the access characteristics of Parquet-based data lakes to explain why search-based workloads can be challenging. We also describe existing approaches commonly taken to address those challenges.

A. Parquet Data Lakes on Object Storage

Organizations have started using object-storage-based data lakes based on Parquet files as the single source of truth for enterprise data, due to their wide ecosystem integrations, compelling cost characteristics and transactional semantics. As depicted in Figure 1, operational data is first ETLed into the data lake from OLTP databases, CRMs, and custom connectors. Once the data lands in the data lake, a query engine such as Trino or SparkSQL is used to query the data lake directly for analytical workloads.

Parquet files are well optimized for OLAP workloads against object storage. A Parquet file can consist of multiple row groups, where each row group in turn consists of a collection of column chunks. A column chunk in a row group consists of all the values of a column in the row range corresponding to the row group. Min-max statistics on the column chunks assist query engines in performing predicate pushdown, reading only column chunks necessary for a particular query. These column chunks, which range from several to hundreds of MB in size, can be downloaded in parallel via byte-range requests against object storage. With VM instance bandwidth reaching 100Gbps in major public clouds, analytics on Parquet files residing in object storage has reached interactive speeds.

B. Search Workloads

While data lakes are designed for SQL OLAP use cases, they still fall short in emerging search-oriented workloads, such as:

- **High-cardinality filtering**, such as filtering on a UUID column. Examples include observability workloads (filtering by Kubernetes pod name) and blockchain analytics (filtering by transaction hash).
- **Substring search**, such as log search or text analytics. For example, to detect if evaluation datasets are leaked in the pretraining corpus, one could perform a substring search against the training records, which might be stored as a text column in a data lake.
- **Vector embedding search**, i.e. approximate nearest neighbor search. Examples include retrieval-augmented-generation (RAG) and recommendation systems.

While query engines can efficiently use Parquet-based data lakes for OLAP workloads, they struggle to do so on these new workloads. There are two main issues:

1) **Useless indices.** In search workloads, it is often infeasible to keep the data sorted based on a high-cardinality column and maintain insertion performance. For example, time series data such as IT monitoring and blockchain logs are naturally sorted by timestamp, and sorting them based on UUID tags amount to an extremely expensive transpose. Natural sort-orders do not exist for text or vector embeddings. These two factors render min-max indices on column chunks useless for search workloads.
2) **Read granularity.** Even if an appropriate index exists, column chunks corresponding to text or binary data types typically tend to be tens to hundreds of MBs in size due to Parquet's design (further explored in Section V-A). This means for highly selective search queries, current query engines have to retrieve many MBs of data from object storage just to return a result that is tens of bytes.

"Lakehouse" query engines today like Spark, Trino, or DuckDB today typically have to perform expensive full-column scans which read GBs to TBs of data to perform search queries, making them very expensive.

C. Existing Approaches

Today practitioners commonly employ two approaches to tackle such search workloads, copy data (ETL) and brute force scan. We also mention an emerging third approach, which is new data formats.

1) Copy Data: Practitioners today commonly copy data from the data lake back into specialized systems tailor-made for their workload, such as Clickhouse [15], ElasticSearch [16], vector databases like Qdrant [17] (Figure 1). While this approach guarantees the best performance for those queries by leveraging specialized systems, it largely negates the benefits of the data lake for the data in question. These systems tend to be compute-storage integrated and are expensive to manage. In addition, the data pipelines duplicate data and reintroduce data consistency and staleness issues that data lakes seek to address in the first place.

While we believe that this is the best approach if the search queries are frequent or require strict latency SLAs, e.g. a search engine like Google, we observe that many important workloads do not have such requirements. For example, an LLM pretraining data exploration system might serve only up to 100 internal users while query latencies up to a minute are acceptable. Similarly, a log management system might serve only a particular SRE team where query latencies up to a few seconds are acceptable.

Authorized licensed use limited to: The University of Utah. Downloaded on January 17,2026 at 03:38:49 UTC from IEEE Xplore. Restrictions apply.

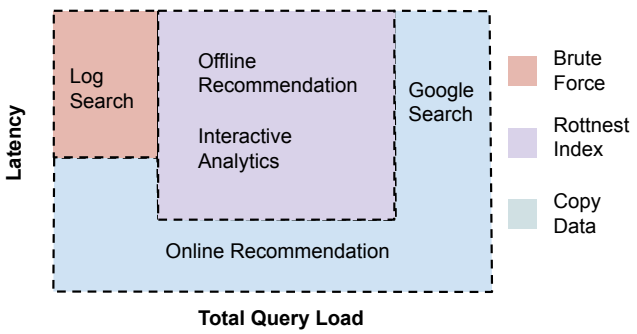


Fig. 2. The most economical approach given latency and throughput requirements of the application. Some example applications are shown.

2) Brute Force Scan: Another popular approach is to simply pay the high IO and compute cost of these queries by horizontally scaling the query engine, such as with Spark SQL [13], [18]. Cloud vendors are especially incentivized to take this approach as they typically charge based on the amount of data scanned. For example, AWS Athena has been heavily used by cybersecurity professionals to query logs, with a pricing model that reflects the brute-force scanning approach.

This approach is the simplest and most economical if querying is highly infrequent, since it incurs no upfront indexing or storage costs. However, it simply defers those costs to query time. While we believe this may be the best approach for workloads that might not ever read most of the data (e.g. log analytics), if most data will be read at least a couple of times over its lifetime, this quickly becomes expensive. In addition, this approaches incurs a minimum latency of typically a few hundred milliseconds due to its need to spin up tasks on demand and access object storage, making it unsuitable for worklaods with strict latency SLAs.

3) New File Format: A third approach is to replace Parquet files with new file formats, such as Lance and Nimble [19], [20]. While these new file formats are purposefully designed to excel at these new search-style workloads, they have not yet achieved the widespread ecosystem integrations that made Parquet appealing in the first place. For example, all three major data lake formats only support Parquet today [2], [3], [5]. As a result, it is impractical today for most organizations to adopt them as the ground truth data store. This means in practice organizations still have to keep data in Parquet, making this no different than the copy-data approach.

Another similar approach is recent modifications to the Parquet file itself, for example column indices and bloom filters [21], [22]. While these approaches could be somewhat effective in some search queries that involve high cardinality tags, they do not help for text or vector searches. More importantly, they assume a Parquet writer that generates these additional indexing structures, which in practice is quite rare.

D. Motivation for a New Approach

The limitations of existing approaches suggest the need for a new design that simultaneously satisfies:

- **Lower indexing and storage cost** compared to copying data into dedicated indices that require expensive persistent RAM/disk resources.
- **Cheaper querying** compared to the brute force approach.

Satisfying these two objectives would provide the most economical solution for a large class of offline or human-interactive workloads with medium total query load, some examples of which are shown in Figure 2. Our objective of easy integration with existing data lakes to reduce data silos leads us to two more design goals:

- **Non-interference guarantee** with existing data lake operations. As the data lake lives at the center of an organization's data architecture, the system must not impact the performance or complexity of existing data lake ingestion, maintenance, or read workloads.
- **Broad compatibility** with all major cloud providers, which might offer varying consistency properties for their object storage offerings.

III. Rottnest Search Indices

Rottnest addresses these four objectives by ***maintaining lightweight index files on top of data lakes.*** Inspired by how data lakes like Delta Lake or Iceberg achieve ACID properties on top of Parquet files in object storage by maintaining log files [3], [4], [23], we propose a novel lazy indexing protocol that constructs secondary indices on data lakes which satisfies the latter two design goals.

Instead of updating indices upon write, Rottnest maintains consistency with the underlying data lake on-demand. Rottnest index maintenance and querying can be performed completely independently of other data lake operations. This approach shares similarities with Postgres concurrent indices and background compactions in LSM-based databases like RocksDB [24], [25]. Like RocksDB, this means the newest data often remains unindexed and requires full scanning during queries. While our protocol is conceptually similar to Azure Hyperspace [26], a key difference lies in the consistency primitives used. Hyperspace relies on atomic rename operations, which are not universally supported across major cloud providers like AWS S3. In contrast, Rottnest's protocol requires only strong read-after-write consistency, making it compatible with all major cloud object stores.

Building on our novel lazy indexing protocol, Rottnest's overall search architecture satisfies our first two design goals: minimizing both index construction/storage costs and query costs. Optimizing along these two objectives allows Rottnest to maximize the area of the purple-shaded area of Figure 2 where it wins over the brute force and copy data approaches. In the red-shaded area, brute force scanning requires no indexing cost but incurs high search costs - lower index costs help Rottnest reach its break-even point with fewer queries. In the blue-shaded area, dedicated systems like ElasticSearch require high upfront costs to store indices in expensive persistent SSDs/memory but offer cheap searches - lower query costs allow Rottnest to remain competitive at higher query loads while also meeting stricter latency requirements.

Authorized licensed use limited to: The University of Utah. Downloaded on January 17,2026 at 03:38:49 UTC from IEEE Xplore. Restrictions apply.

Table Discovery in Data Lakes: State-of-the-art and Future Directions

Grace Fan
Northeastern University
fan.gr@northeastern.edu

Jin Wang
Megagon Labs
jin@megagon.ai

Yuliang Li
Megagon Labs
yuliang@megagon.ai

Renée J. Miller
Northeastern University
miller@northeastern.edu

ABSTRACT

Data discovery refers to a set of tasks that enable users and downstream applications to explore and gain insights from massive collections of data sources such as data lakes. In this tutorial, we will provide a comprehensive overview of the most recent table discovery techniques developed by the data management community. We will cover table understanding tasks such as domain discovery, table annotation, and table representation learning which help data lake systems capture semantics of tables. We will also cover techniques enabling various query-driven discovery and table exploration tasks, as well as how table discovery can support key data science applications such as machine learning and knowledge base construction. Finally, we will discuss future research directions on developing new table discovery paradigms by combining structured knowledge and dense table representations, as well as improving the efficiency of discovery using state-of-the-art indexing techniques, and more.

CCS CONCEPTS

• **Information systems → Top-k retrieval in databases**; *Mediators and data integration*; *Database design and models.*

KEYWORDS

Dataset Discovery, Data Lake, Data Integration, Unionable Tables

ACM Reference Format:
Grace Fan, Jin Wang, Yuliang Li, and Renée J. Miller. 2023. Table Discovery in Data Lakes: State-of-the-art and Future Directions. In *Companion of the 2023 International Conference on Management of Data (SIGMOD-Companion '23), June 18–23, 2023, Seattle, WA, USA.* ACM, New York, NY, USA, 7 pages. https://doi.org/10.1145/3555041.3589409

1 INTRODUCTION

We have witnessed in the last decades a drastic growth in the number of open and shared datasets coming from governments, academic institutes, and private companies. These massive collections of structured or unstructured datasets, which we refer to as *data lakes*, open up new opportunities for innovation, economic growth, and social benefits [39]. As of 2022, the size of US open data (data.gov) has reached a total of 335k datasets contributing to $3 trillion of the US economy [9]. WebDataCommons [30] has also made 233M tables extracted from the open web publicly available. Data lakes store a wide variety of open-domain knowledge from sources such as Wikipedia, news articles, and other online sources, as well as data that is private to an organization or purchased from data brokers. However, while data lakes are invaluable data sources, their large size and complexity can make it difficult for users to find and access the specific data that is required by specific downstream applications. To address this issue, the data management community has been building *data discovery systems* [3, 6, 18, 20, 34, 40, 48], with table discovery from data lakes as the primary application. Data discovery systems help users quickly and easily explore and analyze large datasets, enabling them to gain insights and knowledge that can support decision making and other activities.

This tutorial aims at providing a comprehensive overview of the most recent developments from the database community that enable the aforementioned data discovery methodologies and systems. While there are some previous studies about discovery from other data formats, such as images and JSON [36, 55], we focus on the discovery from tabular data, which is a primary data format in data lakes. We will organize our tutorial by taking an architectural view of table discovery systems depicted in Figure 1. The architecture highlights key system components (in green) and sub-tasks (in blue) that are essential building blocks of an end-to-end data discovery pipeline from raw data lake tables to end-users and downstream applications.

Firstly, we will have an overview of several table understanding tasks. Given an input data lake, table discovery systems typically enrich the raw data lake tables via automatically annotating and indexing techniques to enable better understanding of table semantics. Annotations such as semantic column types can be further used to improve the performance of downstream applications (e.g., table search). We will focus on the tasks of (1) *table annotation* [34] for generating useful meta-data about tables, (2) *domain discovery* [33, 45] for identifying novel domains beyond standard DB data types, (3) machine learning techniques for learning *embeddings* [14, 50] of data items in tables (including columns, rows, and entire tables).

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
SIGMOD-Companion '23, June 18–23, 2023, Seattle, WA, USA
© 2023 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-1-4503-9507-6/23/06…$15.00
https://doi.org/10.1145/3555041.3589409

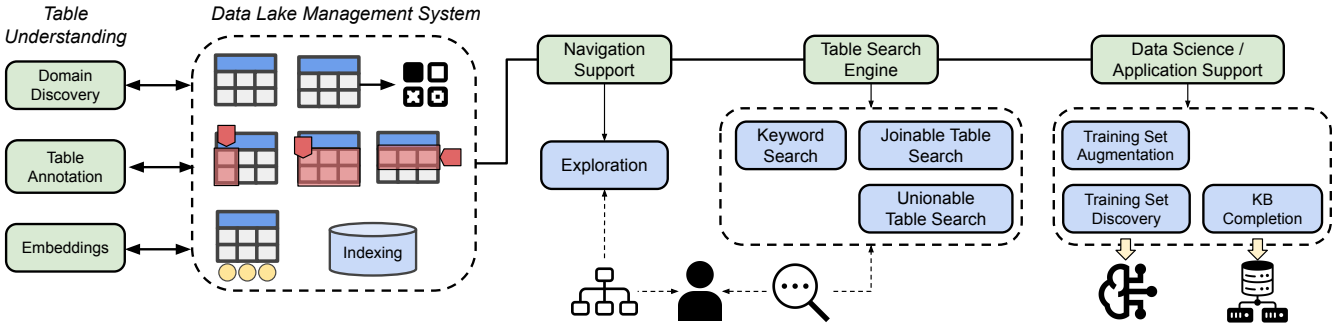


Figure 1: Overview of table discovery in data lakes.

The table discovery system serves the data lake tables to the end-users (e.g., data scientists) by enabling them to query and explore tables that are relevant to their downstream analytic tasks. This is typically achieved via *query driven discovery* [43]: given a user query, retrieve the top-k relevant tables from the data lake. Alternatively, some discovery systems support *data navigation* allowing a user to navigate through the tables in a data lake using a (discovered) organization for the tables [42] or by exploring individual tables followed by tables related through a foreign key or other relationships [18]. And query and navigation can be combined to create organizations over the set of tables retrieved by search [46] in an online manner.

For query driven discovery, we will survey the classic query types includingkeyword search or meta-data search, which often use table schema including attribute names [4, 47, 54] as supported by popular systems such as Google dataset search [3, 21]. Importantly, rather than keywords or table schema, an emerging trend in table discovery systems is to support querying data lakes using table values (e.g., columns or an entire table). Previous studies have identified two types of important search queries: joinable table search [12, 15, 16, 48, 59, 60] and unionable table search [2, 12, 17, 27, 44]. Joinable table search aims at augmenting a query table with additional attributes, while unionable table search is motivated by the need to find tables with new tuples to extend a query table. We will be covering the classic schema-based and value-based approaches, as well as the latest approaches based on knowledge graphs [27] and pre-trained language models [17] that achieve state-of-the-art performance.

We will also cover an emerging trend of performing data discovery to support a variety of data science tasks. One typical example is the creation and augmentation of training datasets. High-quality training datasets are often times a prerequisite for the success of ML models. Data lakes provide a unique opportunity for harvesting training data. Among those techniques, we will primarily introduce (1) data augmentation [8] that aims at finding new features to improve the ML models' performance and (2) training set discovery/construction [58] which focuses on constructing training data from a fixed set of tables via learning representations.

Finally, we will present challenges and future research directions for table discovery. We will cover four main aspects. First, for table understanding which is core to table discovery, we will discuss the possibility of combining two lines of already successful approaches: *knowledge* based approaches [27, 53] and *machine learning* based approaches [14, 17, 23]. We will challenge the community to find synergies between the two approaches by bringing together both rich structured knowledge from KB's and highly effective dense representations from ML. Second, we will further discuss the challenge of boosting the scalability of discovery systems, especially the issues related to indexing. We will elaborate on why indexing is still a challenging unsolved problem for data lakes and how some recently developed techniques on data structures [25], sketching [48], and vector indices [38] can potentially help. Finally, we will discuss the challenge of query-time table annotation and a potential research direction for connecting table discovery with graph mining.

2 TUTORIAL OUTLINE

In this section, we give an overview of the topics that will be covered in the tutorial. They are depicted in Figure 1 as the main components of a data discovery system (in green) and applications (in blue) that are related to table discovery.

2.1 Overview

A popular methodology for table discovery is query-driven discovery [12, 43, 60], where the user starts with a query and aims to find dataset(s) relevant to the query. The query could be either keywords or tables, while the query type could be exact match, joinable, unionable, etc. Existing studies support keyword search over the metadata of tables for users who intend to find tables relevant to a topic (Section 2.3). However, the schemas of tables in data lakes are often unreliable, either missing or incomplete often due to data being shared in primitive formats like CSV [43]. Even when metadata is available, the tables often come from a huge variety of sources meaning the metadata may be inconsistent using different vocabularies and naming conventions. Thus, two important lines of work have emerged. The first we call *Table Understanding*, which are tasks that seek to recover some of the semantics of tables. The second is data-driven table discovery based on a query table (rather than metadata), specifically joinable table search and unionable table search. Yet, data-driven table discovery still has ambiguity in the definitions, which have evolved over time. For example, early work [12] defines related tables as schema-complements and entity-complements based on a defined subject attribute that explains

the table entities. Valentine [28] uses four different notions of joinability and unionability depending on the alignment between the schema of returned tables with that of the query table. Bogatu et al. [2] simultaneously finds joinable and unionable tables by employing five different metrics. Thus, definitions of finding related tables can be very broad, leading to many variations of joinable and unionable table search, which are discussed in Sections 2.4 and 2.5, respectively. Also, there is another aspect of table discovery called navigation [46], which can return a collection of tables in a hierarchical structure that includes not only the contents but also the relationship of tables in the results.

Besides, the outcome of table discovery can benefit many related data science applications. For instance, in the problem of tabular data classification, joinable table search [15] can help extract richer features from the data lakes to enrich the original tables in the training data. And table stitching [29] has been proven rather useful in supporting knowledge base completion.

2.2 Table Understanding

An early influential table discovery system [34] makes use of a variety of table annotations to perform search. These include annotating cells with entities (from an ontology) that the cell mentions, annotating columns with ontology types, and annotating pairs of columns with ontology relationships [34]. The approach uses a supervised probabilistic graphical model to collectively learn all types of annotations. Venetis et al. [51] achieves similar goals by leveraging a proprietary ontology to label columns and binary relationships. More recent work has used labeled training data and supervised learning to label columns with a set of types. These studies start from a set of tables annotated with a fixed set of 78 types and train a machine learning model to apply on unlabeled tables. Sherlock [23] uses a feature-based method to learn column representations for semantic type detection; while Sato [56] makes further improvements by incorporating topic modeling methods (that use row context) for semantic type detection.

Domain discovery is a related problem that aims to identify sets of terms that represent instances of a semantic concept or domain from a given collection of tables. Rather than annotating columns with types, domain discovery collects all values that belong to the same domain. Existing solutions [33, 45] are unsupervised and leverage co-occurrence information to cluster domain terms. In addition Li et al. [33] select a representative as the domain or semantic type.

Recently there is a new trend of utilizing pre-trained language models like BERT to support table annotation applications [14, 50, 52]. The idea is to sequentialize tabular data items by inserting special tokens and use this representation of tables as input to perform fine-tuning over pre-trained language models. This has been done for a variety of annotation applications, including column type detection, relation extraction, cell filling, row population and schema augmentation [14]. Notably, such studies achieve state-of-the-art performance on a variety of benchmarking tasks due to the superior power of language models. We will revisit this observation when we talk about future challenges (Section 3).

2.3 Keyword Search

For users who intend to find tables about a certain topic, some data discovery systems support keyword search to satisfy this need. Keyword search was first used to find tuples within databases [10, 22, 26], but this soon developed into keyword search to find relevant tables, especially in a data lake setting. In this case, a user provides topic keywords and the data discovery system finds relevant tables where different systems have defined relevance by proposing different similarity measurements. The OCTOPUS [4] system supports keyword search over web tables using document keyword search to find pages and then computes the correlation between the keywords and each element (cell) in any table extracted from the document. OCTOPUS returns clusters of tables that share the same schema. In addition, it infers additional attributes by joining tables. In contrast, Pimplikar and Sarawagi [47] use keyword queries describing columns and identify a mapping between the columns of a table and query keywords using table headers, the text surrounding a table on a webpage, and the data within a table. To better facilitate keyword search, Google Dataset Search [3, 21] focuses on integrating and normalizing metadata of tables from various data sources. It works on cleaning, standardizing, and inferring the metadata of each dataset and the relationships among different datasets, while organizing them into a knowledge graph to facilitate retrieval. Keyword search is done only over the metadata, not the data within a table.

2.4 Joinable Table Search

Except for keyword search, many table discovery systems use the data values themselves to find tables that are related to the column values of the given query table. One problem that several approaches aim to solve is joinable table search, with the goal of finding relevant tables to join and augment the query table with additional attributes. InfoGather [54] allows augmentation of a query table with new attributes by first performing schema matching and then either augmentation by attribute names or example tuples, or discovery of related attributes to a given set of entities. Das Sarma et al. [12] defines joinable tables as schema-complements, where schema matching is conducted using attribute names, types, and values to find joinable attributes and non-matching attributes to augment with the query. Lehmberg et al. [31] conducts joinable table search on web tables with an aligned subject attributes by measuring the Jaccard similarity between the data values, metadata, and table contexts.

Although early work tended to use Jaccard similarity to measure the syntactic similarity between attributes, Jaccard has been shown to be bias attributes with smaller cardinality [1]. This is impractical in the case of data lake tables, where there is a large skew in the domain sizes. LSH Ensemble [60] studies the problem of domain search to solve the problem of joinable table search. For domain search, it uses set containment to find attributes with high overlap (containment) in values. In this way, it finds joinable tables by finding domains that highly contain the query domain. By employing a Locality Sensitive Hashing (LSH) index [13] and a data partitioning scheme that optimally minimizes false negatives, LSH ensemble is able to find joinable tables on a large scale even among attributes with skewed cardinality distributions. This line of work continues